

Auditor Léxico de Telemetria de Sensores Industriais

Um firewall léxico construído com Expressões Regulares e AFNε

Linguagens Formais e Autômatos · Trabalho do 1º Bimestre · CESUPA

Augusto Rodrigues · Caue Jadão · César Augusto

github.com/cauebarroso/Auditor-de-sensores-industriais-com-express-es-regulares

O problema e a solução

- Sensores e atuadores conversam com o supervisor (SCADA/IloT) por pacotes de texto.
- Um único símbolo errado vira uma leitura falsa ou uma ordem perigosa.
- Solução: um firewall léxico. O pacote só passa se pertencer à linguagem de uma das cinco ERs.
- Pacotes bloqueados recebem a causa provável e a coluna exata do erro, obtida simulando o AFNε.

Pacotes que chegam à rede

SEN-TMP-0101:TEMP=78.4C	✓
CMD ATU-VLV-0201 AJUSTAR 40%	✓
10.20.0.0/16	✓
2026-09-30T06:42:10 WARN SEN-TMP-0101: ...	
SEN-TMP-0101:TEMP=104.7C	X
letra O no lugar do zero	
CMD ATU-VLV-0201 AJUSTAR 120%	X
ajuste acima de 100%	
2026-09-31T07:10:00 INFO ...	X
setembro tem 30 dias	

Entradas → processamento → saídas

Entradas

- Pacote digitado (modo interativo)
- Pacote por argumento (--pacote)
- Arquivo .txt em lote, com comentários #

Processamento

- Normaliza (remove espaços das pontas)
- re.fullmatch nas 5 ERs
- Extrai campos dos válidos
- Diagnostica os inválidos pelo AFNε

Saídas

- [OK] / [ERRO] com coluna do erro
- Relatório de conformidade
- Filtros por categoria
- Exportação JSON, CSV ou TXT

Módulo	Responsabilidade
expressoes.py	As 5 ERs e as fichas (alfabeto, linguagem, ER formal)
afn.py · construtor_afn.py	AFNε: fecho-ε, simulação, construção de Thompson, prova de equivalência
diagnostico.py · relatorio.py · auditor.py	Diagnóstico, relatório/exportação e interface (menu e linha de comando)

Notação formal × sintaxe no código

Notação formal	No Python	Significado
$r s$	<code>rs</code>	Concatenação
$r \mid s$	<code>r s</code>	União
r^*	<code>r*</code>	Fecho de Kleene (zero ou mais)
$r r^*$	<code>r+</code>	Fecho positivo (uma ou mais)
$(r \mid \epsilon)$	<code>r?</code>	Opcionalidade
$r r r r$	<code>r{4}</code>	Repetição exata
$r r (r \mid \epsilon)$	<code>r{2,3}</code>	Repetição limitada = <code>rr</code> <code>rrr</code>
$[\emptyset-9]$	<code>[0-9]</code>	Classe finita: $(0 \mid 1 \mid \dots \mid 9)$
<code>_ e .</code>	<code>' ' e \.</code>	Espaço e ponto literais (o ponto do Python precisa de escape)
$\langle \text{NOME} \rangle = r$	<code>—</code>	Definição regular: só um nome para a subexpressão

Sempre `re.fullmatch` (dispensa `^` e `$`). Proibidos e rejeitados pelo nosso analisador: `\d \w \s`, ponto curinga, classes negadas, retroreferências e lookarounds.

Identificador de um sensor (SEN) ou atuador (ATU); bloco reutilizado nas ERs 02, 03 e 05.

$\Sigma_1 = \{A, \dots, Z\} \cup \{0, \dots, 9\} \cup \{-\}$ (37 símbolos)

ER formal

$\langle ID \rangle = (SEN \mid ATU) - [A-Z][A-Z]([A-Z] \mid \epsilon) - [0-9][0-9][0-9][0-9]$
ER01 = $\langle ID \rangle$

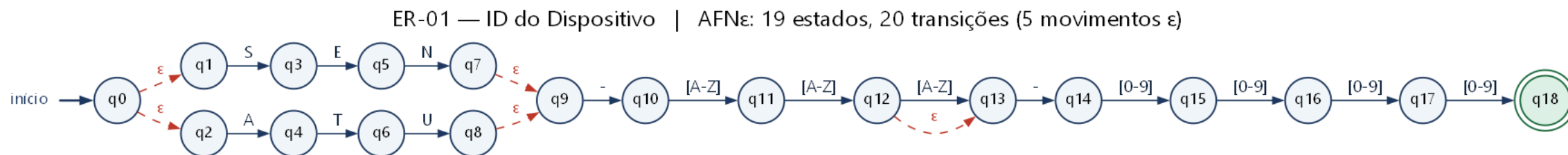
No código (re.fullmatch)

`r"(SEN|ATU)-[A-Z]{2,3}-[0-9]{4}"`

No código	Na ER formal	Operador
<code>(SEN ATU)</code>	$(SEN \mid ATU)$	União
<code>[A-Z]</code>	$(A \mid B \mid \dots \mid Z)$	Classe finita
<code>[A-Z]{2,3}</code>	$[A-Z][A-Z]([A-Z] \mid \epsilon)$	Repetição limitada: rr rrr
<code>[0-9]{4}</code>	$[0-9][0-9][0-9][0-9]$	Repetição exata

ER-01 — AFN ϵ e testes (8 aceitas, 8 rejeitadas)

L = Cadeias formadas pela classe do dispositivo (SEN ou ATU), um hífen, o código do modelo com 2 ou 3 letras maiúsculas, um hífen e o número de série com exatamente 4 dígitos. L_1 é FINITA: $|L_1| = 2 \times (26^2 + 26^3) \times 10^4 = 365.040.000$ cadeias.



#	Cadeia	Esperado	Justificativa
1	SEN-TM-0001	✓ aceita	Sensor com modelo de 2 letras
2	ATU-VLV-0420	✓ aceita	Atuador com modelo de 3 letras
3	SEN-ABC-1234	✓ aceita	Sensor com modelo de 3 letras
4	ATU-XY-9999	✓ aceita	Maior número de série (9999) (caso-limite)
5	SEN-XX-0000	✓ aceita	Menor número de série (0000) (caso-limite)
6	SEN-PH-0750	✓ aceita	Sensor de pH
7	SEN-AA-0000	✓ aceita	Menor cadeia de L_1 (11 símbolos) (caso-limite)
8	ATU-ZZZ-9999	✓ aceita	Maior cadeia de L_1 (12 símbolos) (caso-limite)
9	ϵ (cadeia vazia)	✗ rejeitada	Cadeia vazia ϵ não pertence a L_1 (caso-limite)
10	sen-tm-0001	✗ rejeitada	Minúsculas não pertencem a Σ_1
11	SEN-TM-00012	✗ rejeitada	5 dígitos: {4} exige exatamente 4 (caso-limite)
12	SEN-TM-123	✗ rejeitada	Apenas 3 dígitos no número de série (caso-limite)
13	ATU-A-1234	✗ rejeitada	Modelo com 1 letra (mínimo 2) (caso-limite)
14	SEN-ABCD-1234	✗ rejeitada	Modelo com 4 letras (máximo 3) (caso-limite)
15	SENO-TM-0001	✗ rejeitada	Prefixo SENO não pertence a (SEN ATU)
16	SEN_TM_0001	✗ rejeitada	Separador '_' não pertence a Σ_1

Leitura de um sensor: temperatura (C), umidade (%) ou pressão (hPa), dentro da faixa.

$\Sigma_2 = \{A, \dots, Z\} \cup \{0, \dots, 9\} \cup \{-, :, =, ., \%, h, a\}$ (43 símbolos)

ER formal

```
<SID> = SEN - [A-Z][A-Z]([A-Z] | ε) - [0-9][0-9][0-9][0-9]
<DEC> = . [0-9] | ε
<TEMP> = TEMP= (- | ε) (1[0-9][0-9] | ([1-9] | ε) [0-9]) <DEC> C
<UMID> = UMID= (100 (.0 | ε) | ([1-9] | ε) [0-9] <DEC>) %
<PRES> = PRES= (8[5-9][0-9] | 9[0-9][0-9] | 10[0-9][0-9]) hPa
ER02 = <SID> : (<TEMP> | <UMID> | <PRES>)
```

No código (re.fullmatch)

```
r"SEN-[A-Z]{2,3}-[0-9]{4}:(TEMP=-?(1[0-9][0-9]|[1-9]?[0-9])(\.[0-9])?C|UMID=(100(\.0)?|[1-9]?[0-9](\.[0-9])?)%|PRES=(8[5-9][0-9]|9[0-9][0-9]|10[0-9][0-9])hPa)"
```

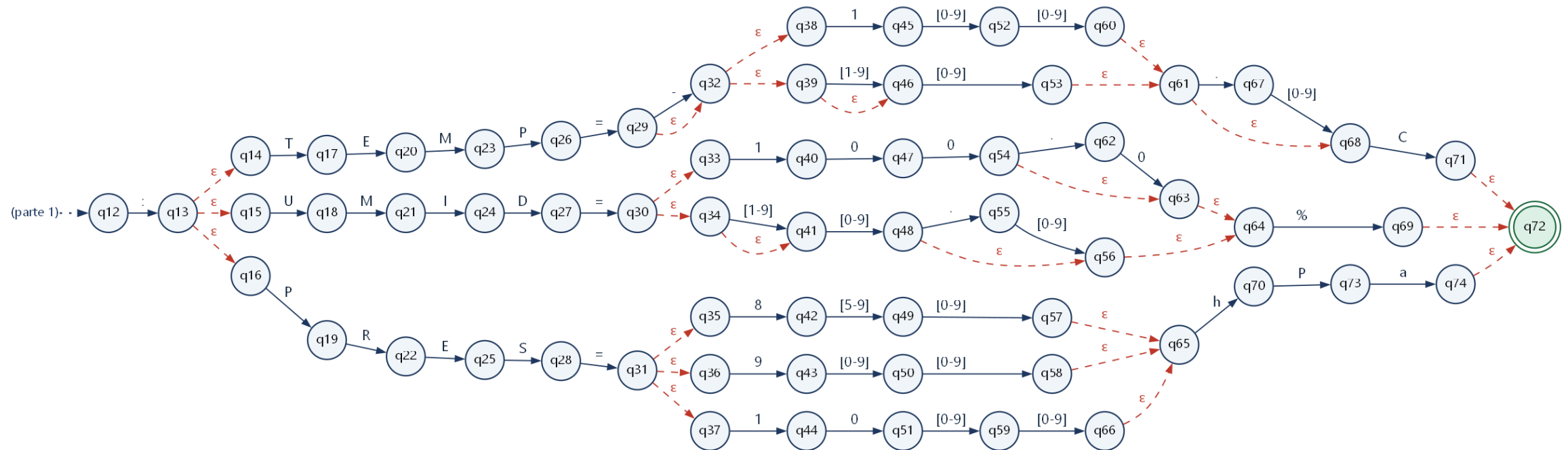
No código	Na ER formal	Operador
-?	(- ε)	Opcionalidade
[1-9]?[0-9]	([1-9] ε)[0-9]	0 a 99 sem zero à esquerda
(\.[0-9])?	.[0-9] ε	Escape do ponto + opcional
(TEMP=... UMID=... PRES=...)	<TEMP> <UMID> <PRES>	União de três ramos

ER-02 — AFN ϵ : o ID do sensor e a união das três medições

ER-02 — parte 1 de 2



ER-02 — parte 2 de 2



ER-02 — linguagem e testes (8 aceitas, 8 rejeitadas)

L = ID de um SENSOR (SEN), ':' e exatamente uma medição: TEMP= com sinal '-' opcional, inteiro de 0 a 199 sem zeros à esquerda, uma casa decimal opcional e a unidade C; UMID= com valor de 0 a 100 (uma casa decimal opcional; 100 só como 100 ou 100.0) e a unidade %; ou PRES= com inteiro de 850 a 1099 e a unidade hPa. L₂ é finita.

#	Cadeia	Esperado	Justificativa
1	SEN-TM-0001:TEMP=25.5C	✓ aceita	Temperatura típica com uma casa decimal
2	SEN-TM-0001:TEMP=-10C	✓ aceita	Temperatura negativa sem casa decimal
3	SEN-TMP-0002:TEMP=-199.9C	✓ aceita	Menor temperatura da faixa (caso-limite)
4	SEN-TM-0002:TEMP=199.9C	✓ aceita	Maior temperatura da faixa (caso-limite)
5	SEN-AB-1234:UMID=100.0%	✓ aceita	Umidade máxima (100.0) (caso-limite)
6	SEN-AB-1234:UMID=0%	✓ aceita	Umidade mínima (0) (caso-limite)
7	SEN-YY-0000:PRES=850hPa	✓ aceita	Menor pressão da faixa (caso-limite)
8	SEN-XX-9999:PRES=1099hPa	✓ aceita	Maior pressão da faixa (caso-limite)
9	SEN-TM-0001:TEMP=200C	✗ rejeitada	200 > 199: fora da faixa (caso-limite)
10	SEN-TM-0001:UMID=100.5%	✗ rejeitada	100.5% > 100%: fora da faixa (caso-limite)
11	SEN-TM-0001:PRES=849hPa	✗ rejeitada	849 < 850: fora da faixa (caso-limite)
12	SEN-TM-0001:PRES=1100hPa	✗ rejeitada	1100 > 1099: fora da faixa (caso-limite)
13	SEN-TM-0001:TEMP=25.55C	✗ rejeitada	Duas casas decimais (máximo uma)
14	SEN-TM-0001:TEMP=025C	✗ rejeitada	Zero à esquerda não é permitido
15	ATU-TM-0001:TEMP=25C	✗ rejeitada	Atuador (ATU) não emite telemetria
16	SEN-TM-0001:TEMP=25F	✗ rejeitada	Unidade F não pertence à linguagem (só C)

Ordem para um atuador: ligar, desligar, abrir, fechar ou ajustar de 0 a 100%.

$\Sigma_3 = \{A, \dots, Z\} \cup \{0, \dots, 9\} \cup \{-, _, \%\}$ (39 símbolos)

ER formal

```
<AID> = ATU - [A-Z][A-Z]([A-Z] | ε) - [0-9][0-9][0-9][0-9]
<PCT> = 100 | ([1-9] | ε) [0-9]
ER03 = CMD _ <AID> _ (LIGAR | DESLIGAR | ABRIR | FECHAR | AJUSTAR _ <PCT> %)
```

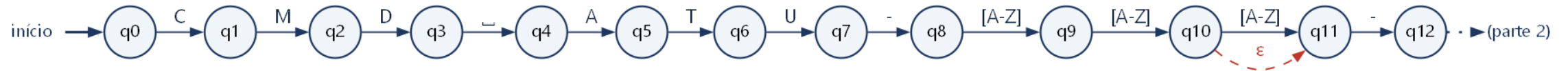
No código (re.fullmatch)

```
r"CMD ATU-[A-Z]{2,3}-[0-9]{4} (LIGAR|DESLIGAR|ABRIR|FECHAR|AJUSTAR (100|[1-9]?[0-9]))%"
```

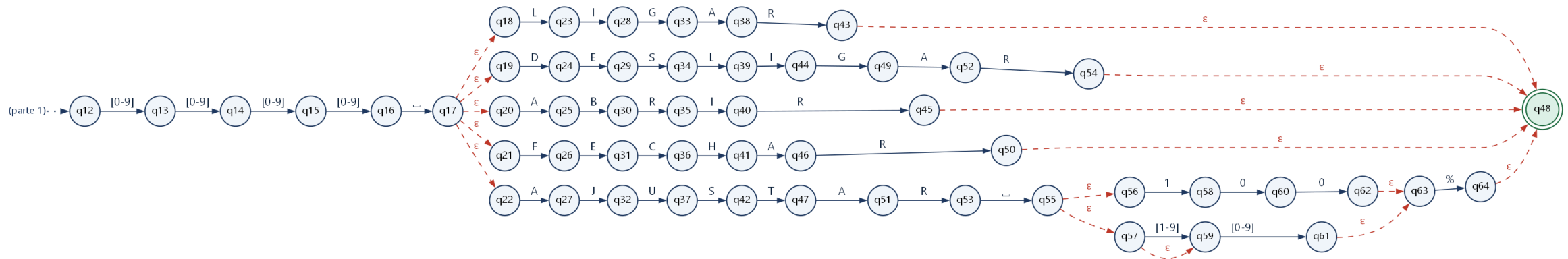
No código	Na ER formal	Operador
(espaço)	_	Símbolo de Σ_3 (não usamos \s)
(LIGAR ... AJUSTAR ...)	LIGAR ... AJUSTAR _ <PCT> %	União de cinco ações
(100 [1-9]?[0-9])	100 ([1-9] ε)[0-9]	União: 0 a 100
{2,3} {4}	rr(r ε) rrrr	Repetições do ID

ER-03 — AFN ϵ : o ID do atuador e as cinco ações

ER-03 — parte 1 de 2



ER-03 — parte 2 de 2



ER-03 — linguagem e testes (8 aceitas, 8 rejeitadas)

L = O prefixo 'CMD', um espaço, o ID de um ATUADOR (ATU), um espaço e exatamente uma ação: LIGAR, DESLIGAR, ABRIR, FECHAR ou 'AJUSTAR ' seguido de um inteiro de 0 a 100 sem zeros à esquerda e do símbolo %. L_3 é finita.

#	Cadeia	Esperado	Justificativa
1	CMD ATU-VLV-0420 LIGAR	✓ aceita	Ação LIGAR
2	CMD ATU-VLV-0420 DESLIGAR	✓ aceita	Ação DESLIGAR
3	CMD ATU-XX-1234 ABRIR	✓ aceita	Ação ABRIR
4	CMD ATU-XX-1234 FECHAR	✓ aceita	Ação FECHAR
5	CMD ATU-YY-9999 AJUSTAR 100%	✓ aceita	Maior ajuste (100%) (caso-limite)
6	CMD ATU-YY-9999 AJUSTAR 0%	✓ aceita	Menor ajuste (0%) (caso-limite)
7	CMD ATU-MTR-0007 AJUSTAR 75%	✓ aceita	Ajuste com 2 dígitos
8	CMD ATU-BB-0001 AJUSTAR 9%	✓ aceita	Ajuste com 1 dígito
9	CMD ATU-VLV-0420 AJUSTAR 101%	✗ rejeitada	101 > 100: fora da faixa (caso-limite)
10	CMD SEN-VLV-0420 LIGAR	✗ rejeitada	Sensor (SEN) não recebe comandos
11	CMD ATU-VLV-0420 PAUSAR	✗ rejeitada	PAUSAR não pertence à união de ações
12	CMD ATU-VLV-0420 AJUSTAR %	✗ rejeitada	Valor percentual ausente
13	CMD ATU-VLV-0420 LIGAR_	✗ rejeitada	Espaço extra no final (caso-limite)
14	CMD ATU-VLV-0420 AJUSTAR 050%	✗ rejeitada	Zero à esquerda não é permitido
15	cmd ATU-VLV-0420 LIGAR	✗ rejeitada	Prefixo em minúsculas
16	CMD ATU-VLV-0420 AJUSTAR 50	✗ rejeitada	Falta o símbolo %

Endereço IPv4 com octetos de 0 a 255, sem zero à esquerda, e máscara CIDR opcional (/0 a /32).

$\Sigma_4 = \{0, \dots, 9\} \cup \{., /\}$ (12 símbolos)

ER formal

```
<OCT> = 25[0-5] | 2[0-4][0-9] | 1[0-9][0-9] | ([1-9] | ε) [0-9]
<MASC> = 3[0-2] | ([12] | ε) [0-9]
ER04 = <OCT> . <OCT> . <OCT> . <OCT> (/ <MASC> | ε)
```

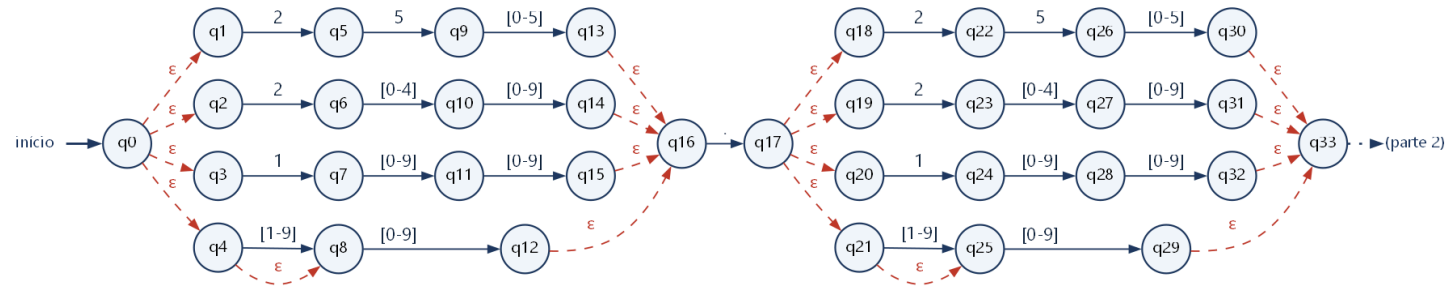
No código (re.fullmatch)

```
r"((25[0-5]|2[0-4][0-9]|1[0-9][0-9]|[1-9]?[0-9])\\.){3}(25[0-5]|2[0-4][0-9]|1[0-9][0-9]|[1-9]?[0-9])(/(3[0-2]|[12]?[0-9]))?"
```

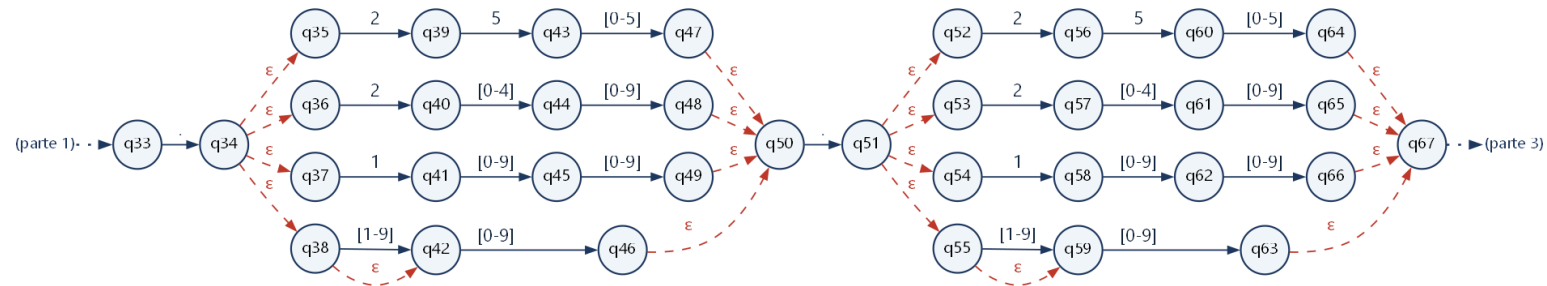
No código	Na ER formal	Operador
25[0-5] 2[0-4][0-9] 1[0-9][0-9] [1-9]?[0-9]	<OCT>	União de 4 faixas disjuntas
(...\.){3}	<OCT>.<OCT>.<OCT>.	Repetição exata de grupo
\.	.	Escape: ponto literal
(/(3[0-2] [12]?[0-9]))?	(/ <MASC> ε)	Máscara opcional

ER-04 — AFN ϵ : quatro octetos e a máscara opcional

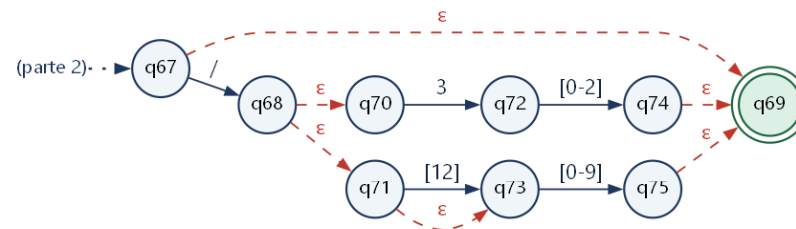
ER-04 — parte 1 de 3



ER-04 — parte 2 de 3



ER-04 — parte 3 de 3



ER-04 — linguagem e testes (8 aceitas, 8 rejeitadas)

L = Quatro octetos decimais de 0 a 255, sem zeros à esquerda, separados por '.', seguidos opcionalmente de '/' e de uma máscara de 0 a 32. L_4 é finita: $256^4 \times (1 + 33)$ endereços/rotas.

#	Cadeia	Esperado	Justificativa
1	192.168.0.1	✓ aceita	IP privado sem máscara
2	10.0.0.1/8	✓ aceita	IP com máscara CIDR
3	255.255.255.255	✓ aceita	Maior octeto em todas as posições (caso-limite)
4	0.0.0.0	✓ aceita	Menor octeto em todas as posições (caso-limite)
5	172.16.254.1/32	✓ aceita	Maior máscara (/32) (caso-limite)
6	0.0.0.0/0	✓ aceita	Menor máscara (/0): rota padrão (caso-limite)
7	192.168.1.0/24	✓ aceita	Rede /24
8	200.249.250.199	✓ aceita	Usa os ramos 2[0-4]D, 25[0-5] e 1DD do octeto
9	256.0.0.1	✗ rejeitada	256 > 255: octeto fora da faixa (caso-limite)
10	192.168.0.1/33	✗ rejeitada	Máscara 33 > 32 (caso-limite)
11	192.168.0	✗ rejeitada	Apenas 3 octetos
12	1.2.3.4.5	✗ rejeitada	5 octetos
13	192.168.01.1	✗ rejeitada	Zero à esquerda no octeto
14	192.168.0.1/08	✗ rejeitada	Zero à esquerda na máscara
15	10.0.0.1/	✗ rejeitada	Barra sem máscara
16	ε (cadeia vazia)	✗ rejeitada	Cadeia vazia ε não pertence a L_4 (caso-limite)

Registro de evento: data válida, hora, severidade, dispositivo e mensagem.

$\Sigma_5 = \{A, \dots, Z\} \cup \{a, \dots, z\} \cup \{0, \dots, 9\} \cup \{-, :, _, \cdot\}$ (67 símbolos)

ER formal

```
<DATA> = [0-9][0-9][0-9][0-9] - (([0-1-9] | 1[0-2]) - ([0-1-9] | [12][0-9])) | ([0-13-9] | 1[0-2]) - 30 | ([0-13578] | 1[02]) - 31)
<HORA> = ([01][0-9] | 2[0-3]) : [0-5][0-9] : [0-5][0-9]
<SEV> = INFO | WARN | ERROR | CRIT
<ID> = (SEN | ATU) - [A-Z][A-Z]([A-Z] | ε) - [0-9][0-9][0-9][0-9]
<PAL> = [A-Za-z0-9_.-] [A-Za-z0-9_.-]*
<MSG> = <PAL> ( _ <PAL> )*
ER05 = <DATA> T <HORA> _ <SEV> _ <ID> : _ <MSG>
```

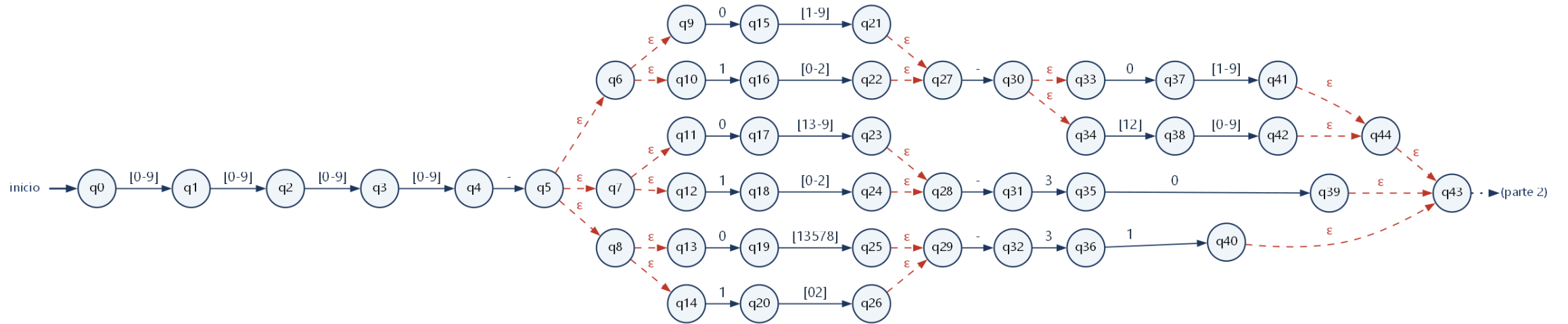
No código (re.fullmatch)

```
r"[0-9]{4}-((0[1-9]|1[0-2])-(0[1-9]|[12][0-9])|(0[13-9]|1[0-2])-30|(0[13578]|1[02])-31)T([01][0-9]|2[0-3]):[0-5][0-9]:[0-5][0-9](INFO|WARN|ERROR|CRIT) (SEN|ATU)-[A-Z]{2,3}-[0-9]{4}: [A-Za-z0-9_.-]+( [A-Za-z0-9_.-]+)*"
```

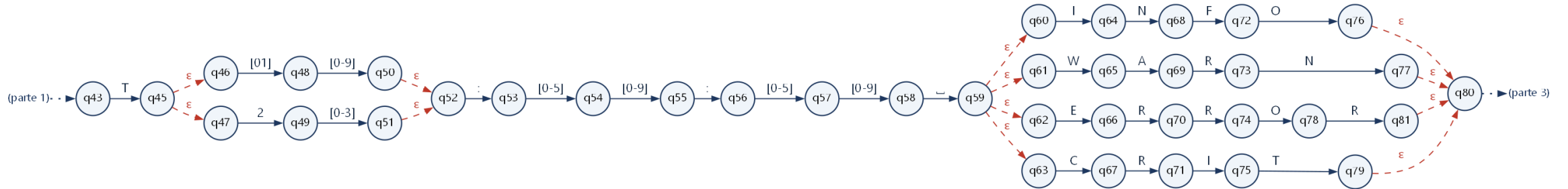
No código	Na ER formal	Operador
(...-(0[1-9] [12][0-9]) ...-30 ...-31)	três casos em <DATA>	União: dia válido por mês
(INFO WARN ERROR CRIT)	<SEV>	União
[A-Za-z0-9_.-]+	<PAL> = c c*	Fecho positivo: r+ = rr*
([A-Za-z0-9_.-]+)*	(_ <PAL>)*	Fecho de Kleene

ER-05 — AFN ϵ da data e da hora

ER-05 — parte 1 de 4



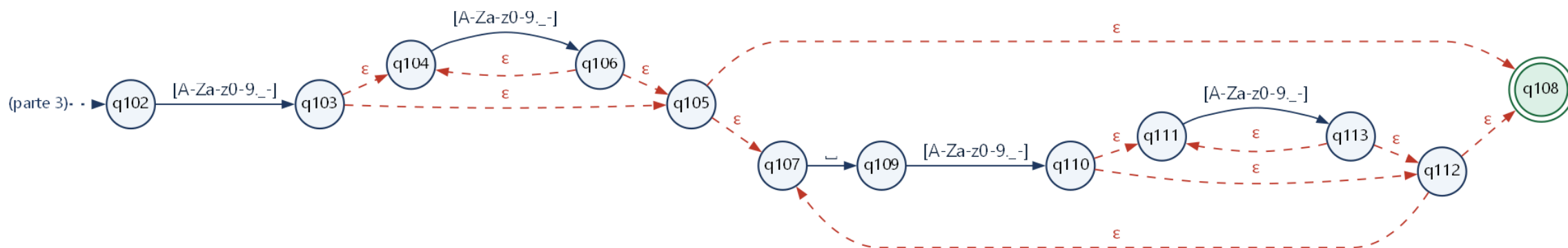
ER-05 — parte 2 de 4



ER-05 — a mensagem usa fecho de Kleene

$\langle \text{PAL} \rangle = [\text{A-Za-z0-9_.-}] [\text{A-Za-z0-9_.-}]^*$ $\langle \text{MSG} \rangle = \langle \text{PAL} \rangle (_ \langle \text{PAL} \rangle)^*$
No código: $[\text{A-Za-z0-9_.-}]^+ ([\text{A-Za-z0-9_.-}]^+)^*$

ER-05 — parte 4 de 4



- Laço interno: o fecho de $[\text{A-Za-z0-9_.-}]^*$ ($q104 \leftrightarrow q106$). Laço externo: $(_ \langle \text{PAL} \rangle)^*$, que volta de $q112$ para $q107$.
- Por causa do fecho, L_5 é a única linguagem INFINITA do projeto; as outras quatro são finitas.
- Rejeita mensagem vazia, dois espaços seguidos e símbolos fora de Σ_5 (ex.: pressão, com ã).

ER-05 — linguagem e testes (8 aceitas, 8 rejeitadas)

L = Data AAAA-MM-DD com dia válido para o mês (fevereiro até 29; abril, junho, setembro e novembro até 30; os demais até 31), 'T', hora hh:mm:ss (00:00:00 a 23:59:59), espaço, nível de severidade (INFO, WARN, ERROR ou CRIT), espaço, ID do dispositivo, ':' e uma mensagem formada por uma ou mais palavras de [A-Za-z0-9_-] separadas por exatamente um espaço. L₅ é INFINITA, pois a mensagem usa o fecho de Kleene.

#	Cadeia	Esperado	Justificativa
1	2026-09-30T08:15:00 INFO SEN-TM-0001: leitura normal	✓ aceita	Registro informativo típico
2	2026-12-31T23:59:59 CRIT ATU-XX-9999: falha critica no motor	✓ aceita	Último segundo do ano (caso-limite)
3	2026-01-01T00:00:00 WARN SEN-AB-1234: bateria fraca	✓ aceita	Primeiro instante do ano (caso-limite)
4	2026-02-28T12:30:45 ERROR ATU-VLV-0420: travamento_123	✓ aceita	Mensagem com _ e dígitos
5	2028-02-29T06:00:00 INFO SEN-TM-0001: ok	✓ aceita	29 de fevereiro (caso-limite)
6	2026-04-30T10:00:00 WARN ATU-VLV-0420: valvula 80.5 aberta	✓ aceita	Dia 30 em mês de 30 dias (caso-limite)
7	2026-07-31T18:45:00 ERROR SEN-PH-0750: sensor-ph fora_de_faixa	✓ aceita	Dia 31 em mês de 31 dias (caso-limite)
8	2026-10-15T14:22:10 INFO SEN-ZZ-0000: .	✓ aceita	Mensagem de 1 símbolo (caso-limite)
9	2026-13-01T10:00:00 INFO SEN-TM-0001: ok	✗ rejeitada	Mês 13 não existe (caso-limite)
10	2026-02-30T10:00:00 INFO SEN-TM-0001: ok	✗ rejeitada	30 de fevereiro não existe (caso-limite)
11	2026-04-31T10:00:00 INFO SEN-TM-0001: ok	✗ rejeitada	31 de abril não existe (caso-limite)
12	2026-09-30T24:00:00 INFO SEN-TM-0001: ok	✗ rejeitada	Hora 24 (máximo 23) (caso-limite)
13	2026-09-30T08:15:00 DEBUG SEN-TM-0001: ok	✗ rejeitada	DEBUG não pertence à união de severidades
14	2026-09-30T08:15:00 INFO SEN-TM-0001: _	✗ rejeitada	Mensagem vazia (caso-limite)
15	2026-09-30T08:15:00 INFO SEN-TM-0001: leitura dupla	✗ rejeitada	Dois espaços seguidos na mensagem
16	2026-09-30T08:15:00 INFO SEN-TM-0001: pressão alta	✗ rejeitada	'ã' não pertence a Σ_5

Como garantimos que é a mesma linguagem

1. Código → AFNε (Thompson)

O AFNε é construído a partir do padrão exato do código.

2. ER formal $\equiv$ código: prova

Busca em largura sobre o produto dos dois AFNε (construção de subconjuntos). Nenhuma cadeia distingue as linguagens.

3. Arquivo .jff $\equiv$ código: prova

O .jff é lido de volta e comparado pelo mesmo algoritmo.

4. Executado no próprio JFLAP 7.1

80 de 80 cadeias de teste com o resultado esperado.

Verificação	Resultado
ER formal $\equiv$ código	5/5 provadas
.jff $\equiv$ código	5/5 provadas
AFNε × re (aleatórias)	10.000 cadeias, 0 divergências
JFLAP 7.1	80/80
Linguagens disjuntas	interseções vazias

Achado: o JFLAP 7.1 trata rótulos com '[' como intervalo ([a-z]) e falha com o símbolo '['. Por isso a severidade é INFO, não [INFO]: o .jff reconhece a linguagem exata, sem símbolos substitutos.

Demonstração

```
python src/auditor.py dados/turno_caldeira.txt          # lote + relatório + filtros
python src/auditor.py --pacote "CMD ATU-VLV-0201 AJUSTAR 120%"
python src/auditor.py --explicar "ATU-VLV-0420"         # simulação do AFNε
JFLAP: automatos/jflap/ER-03.jff → Input › Multiple Run → Load Inputs (entradas/ER-03.txt)
```

[ERRO] ER-03 Comando de Controle: valor de ajuste 120% acima de 100%; o intervalo aceito é 0 a 100%.
CMD ATU-VLV-0201 AJUSTAR 120%

^

Onde: coluna 28: símbolo '0' inesperado; esperado '%'

passo	lido	conjunto de estados ativos (após o fecho-ε)
0	–	{q0, q1, q2}
1	'A'	{q4}
2	'T'	{q6}
⋮		
12	'0'	{q18}

Resultado: ACEITA – o último conjunto contém o estado final (q18).

Testes e resultados

Arquivo	O que verifica	Testes
test_aplicacao.py	Entradas vazias/inválidas, diagnóstico, lote, CLI	66
test_automatos.py	AFN ϵ $\times$ re, provas de equivalência, jff, linguagens disjuntas	123
test_expressoes.py	Cadeias aceitas/rejeitadas, requisitos 6+6, categoria única	130
Total	python -m pytest	319 ✓

Casos-limite cobertos

- 0% e 100%; 850 e 1099 hPa
- octetos 0 e 255; máscaras /0 e /32
- 29/02, 30/04, 31/07 aceitas; 30/02, 31/04, hora 24 rejeitadas
- menor e maior cadeia de L_1 ; a cadeia vazia ϵ

Além das 80 cadeias de teste

- JFLAP 7.1: 80/80 com o resultado esperado
- 10.000 cadeias aleatórias: 0 divergências entre AFN ϵ e re
- Dados de exemplo: 20 íntegros e 14 corrompidos, todos com a causa correta

Limitações e possíveis melhorias

Limitações (falsos resultados)

- A validação é léxica: um ID bem formado passa mesmo se o dispositivo não existir.
- -0C é aceito; uma leitura real acima de 199.9 °C é bloqueada.
- 29/02 é aceito em qualquer ano.
- A mensagem do log não aceita acentos.

Melhorias

- Ano bissexto: também é regular (anos de 4 dígitos formam um conjunto finito), mas o AFNε cresceria muito.
- Faixas configuráveis por tipo de sensor.
- Leitura contínua de porta serial ou socket.
- Minimizar o AFD (Hopcroft) e comparar com o AFNε de Thompson.

Contribuições e uso de IA

Integrante	Contribuições
Augusto Rodrigues	ER-01 e ER-02 (fichas, AFNe, testes); diagnóstico de falhas
Caue Jadão	ER-03 e ER-04; revisão geral, verificação de equivalência, relatório e repositório
César Augusto	ER-05; modo lote, relatório de conformidade, exportação e apresentação

Uso de IA (declarado no relatório e no README)

- Antigravity: versão inicial do código, dos testes e dos primeiros .jff.
- Claude (Anthropic): revisão das ERs e da notação formal, construção de Thompson, verificação de equivalência, testes, relatório e slides.
- As decisões de projeto são da equipe; todos compreendem, explicam e modificam o conteúdo.

Obrigado!

Perguntas?

github.com/cauebarroso/Auditor-de-sensores-industriais-com-express-es-regulares